Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

BIG DATA PROCESSING

ASSIGNMENT 1: SPARK CORE & SPARK DATAFRAMES

BACKGROUND.

From the window you can see the sun shining in a lovely autumn morning. Its Monday, 10am, and you are in the open plan office of a new start-up, OptimiseYourJourney, which will enter the market next year with a clear goal in mind: "*leverage Big Data technologies for improving the user experience in transportation*".

The start-up is at a very early stage, and has no clear product in mind yet. However, they have offered a short-term internship in their Big Data Engineering Department to help them exploring the datasets, technologies and techniques that can be applied in their future products. They do not pay very well (0€ per hour), but you see this as a good opportunity to complement your knowledge in the module Big Data Processing you are studying at the moment, so you have decided to give it a go.

OptimiseYourJourney



In the department meeting that has just finished your boss was particularly happy. During the weekend he was exploring the public datasets provided by the Irish government at https://data.gov.ie/ and he found a transportation dataset named **Dublin Bus GPS sample data from Dublin City Council (Insight Project)**: https://data.gov.ie/dataset/dublin-bus-gps-sample-data-from-dublin-city-council-insight-project The original dataset contains 40+ millions of GPS data measurements collected from Dublin buses operating in January of 2013. Once extracted, it occupies ~5GB and is available at:
http://opendata.dublincity.ie/TrafficOpenData/sir010113-310113.zip

Your boss thinks this dataset provides a great opportunity to explore the potential of Spark Core and Spark DataFrames in analysing large datasets. He has already cleaned the dataset for you to perform some data analysis on it.

DATASET.

The new dataset (obtained after cleaning the original one) is provided to you in the folder **Canvas => 5_Assignments => A01_dataset.zip**.

It occupies ~3GB and contains 744 files, one per hour interval and day of the month:

- siri.201301**01**00.csv => Provides the data measurements of **01st** of January 2013 in the hour interval **[12am, 1am)**
- siri.201301**01**01.csv => Provides the data measurements of **01st** of January 2013 in the hour interval **[1am, 2am)**
- ...
- siri.201301**01**08.csv => Provides the data measurements of **01st** of January 2013 in the hour interval **[8am, 9am)**
- siri.201301**01**09.csv => Provides the data measurements of **01st** of January 2013 in the hour interval **[9am, 10am)**
- ...
- siri.201301**01**23.csv => Provides the data measurements of **01st** of January 2013 in the hour interval **[11pm, 12am)**
- siri.201301**02**00.csv => Provides the data measurements of **02nd** of January 2013 in the hour interval **[12am, 1am)**
- ...
- siri.201301**31**23.csv => Provides the data measurements of **31st** of January 2013 in the hour interval **[11pm, 12am)**

Each row of a file contains the following fields:
Date , Bus_Line , Bus_Line_Pattern , Congestion , Longitude , Latitude , Delay , Vehicle , Closer_Stop , At_Stop

- **(00) Date**
  - A String representing the date of the measurement with format <%Y-%m-%d %H:%M:%S>
  - Example: "2013-01-01 13:00:02"
- **(01) Bus_Line**
  - An Integer representing the bus line.
  - Example: 120
- **(02) Bus_Line_Pattern**
  - A String identifier for the sequence of stops scheduled in the bus line (different buses of the same bus line can follow different sequence of stops in different iterations).
  - Example: "027B1001" (it can also be empty "").
- **(03) Congestion**
  - An Integer representing whether the bus is at a traffic jam (No => 0 / Yes => 1) .
  - Example: 0
- **(04) Longitude**
  - A Float representing the longitude position of the bus.
  - Example: -6.269634
- **(05) Latitude**

- A Float representing the latitude position of the bus.
- Example: 53.360504

- **(06) Delay**
  - An integer representing the delay of the bus with respect to its schedule (measured in seconds). It is a negative value if the bus is ahead of schedule.
  - Example: 90.
- **(07) Vehicle**
  - An integer identifier for the bus vehicle.
  - Example: 33304.
- **(08) Closer_Stop**
  - An integer identifier for the closest bus stop.
  - Example: 7486.
- **(09) At_Stop_Stop**
  - An integer representing whether the bus vehicle is at the bus stop right now (i.e., stopping at it for passengers to hop on / hop off). (No -> 0 and Yes -> 1)
  - Example: 0.

TASKS / EXERCISES.

The tasks / exercises to be completed as part of the assignment are described in the next pages of this PDF document.

- The following exercises are placed in the folder **my_code:**
    1. A01_ex1_spark_core.py
    2. A01_ex1_spark_dataframes.py
    3. A01_ex2_spark_core.py
    4. A01_ex2_spark_dataframes.py
    5. A01_ex3_spark_core.py
    6. A01_ex3_spark_dataframes.py

    Each exercise is worth 3.33 marks.

    7. A01_ex4_spark_core.py
    8. A01_ex4_spark_dataframes.py
    9. A01_ex5_spark_core.py
    10. A01_ex5_spark_dataframes.py
    11. A01_ex6_spark_core.py
    12. A01_ex6_spark_dataframes.py
    13. A01_ex7_spark_core.py
    14. A01_ex7_spark_dataframes.py

    Each exercise is worth 6.25 marks.

    Rules:

    - On each exercise, your task is to complete the function **my_main** of the Python program. This function is in charge of specifying the Spark Job performing the desired data analysis.
    - When programming my_main, you can create and call as many auxiliary functions as you need.
    - Do not alter the parameters passed to the function my_main.
    - The entire work must be done "within Spark":
        - The function my_main must start with a creation operation textFile or read loading the dataset to Spark Core and Spark DatasFrames, respectively.
        - The function my_main must finish with an action operation collect gathering and printing by the screen the result of Spark Core / Spark DataFrames job.
        - The function my_main must not contain any other action operation collect other than the one appearing at the very end of the function.
        - The resVAL iterator returned by collect must be printed straight away, you cannot edit it to alter its format for printing.

- The following exercise is placed in the folder **my_code** and **my_report:**
    15. **my_code/**A01_ex8.py
    - **The use-case over a novel dataset justification, design and implementation is worth 30 marks.**

RUBRIC.

Exercises 1-14.

- 25% of the marks => Complete attempt of the exercise (even if it does not lead to the right solution or right format).
- 18.75% of the marks => Right solution and format (following the aforementioned rules) for the "Small Dataset".
- 18.75% of the marks => Right solution and format (following the aforementioned rules) for the "Complete Dataset".
- 37.5% of the marks => Right solution and format (following the aforementioned rules) for any "Additional Dataset" test case we will generate. The marks will be allocated in a per test basis (i.e., if 2 extra test are tried, each of them will represent 18.75% of the marks).

Exercise 15.

- 25% of the marks => Novel Dataset Description.
- 75% of the marks => Novel Use-Case justification, design and implementation.

TEST YOUR SOLUTIONS.

- The folder **my_results** contains the expected results for each exercise.
    - The file **test_checker.py** needs two files and compares whether their content is equal or not.
    - When you have completed one exercise (e.g., A01_ex1_spark_core.py), run it under one of the test cases suggested at the bottom of the file, so that it automatically fills in the respective file in **my_results/Student_Solutions/**
    - Open the file **test_checker.py** and edit the lines 78 and 79 with the names of your file and the one you are comparing it against.
    - Run the program **test_checker.py**. It will tell you whether your output is correct or not.

Main Message

Use the program **test_checker.py** to ensure that all your exercises produce the expected output, including the right format. If the output of the test_checker is **true**, then you can consider that your Spark implementation is correct. Otherwise, keep working on it.

SUBMISSION DETAILS / SUBMISSION CODE OF CONDUCT.

Submit to Canvas by Friday 6th of November, 5:00pm.

- Submissions up to 1 week late will have 10 marks deducted.
- Submissions up to 2 weeks late will have 20 marks deducted.

On submitting the assignment you adhere to the following declaration of authorship. If you have any doubt regarding the plagiarism policy discussed at the beginning of the semester do not hesitate in contacting me.

Declaration of Authorship

I, ___YOUR NAME___, declare that the work presented in this assignment titled 'Assignment 1: Spark Core and Spark DataFrames' is my own. I confirm that:

- This work was done wholly by me as part of my Msc. in Artificial Intelligence or my Msc. in Software Architecture and Design at Munster Technological University.
- Where I have consulted the published work and source code of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this assignment source code and report is entirely my own work.

EXERCISE 1.

Dublin is such a big city, and so one can expect a large number of bus lines operating in it. A commuter wants to know which bus lines actually stop at their nearest bus stop, so they can judge whether it is worth walking there instead of to another stop more far away. Please note that passing nearby the bus stop is not enough: the bus must be genuinely stopped at it (i.e., for passengers to hop on / hop off). The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- The bus stop of interest "stop_id" (e.g., 279)

Your task is to:

- Compute the distinct bus lines stopping at stop_id (i.e., with measurements reporting Closer_Stop == stop_id and At_Stop == 1). Measurements close to stop_id but not stopped at it (At_Stop == 0) must be discarded.

The format of the solution computed must be:

- < L1, L2, ..., Ln >

where:

- Li is an Integer representing a bus line stopping at stop_id.
- < L1, L2, ..., Ln > are sorted by increasing order of Li.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_1

Let's assume we are interested in "stop_id" = 279.

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-02 08:00:00,**40**,015B1001,0,-6.258078,53.359279,30,33488,**279**,**1**

  This measurement is of interest to us. Line 40 is stopped at bus stop 279.

- 2013-01-02 08:05:00,50,027B1002,0,-6.259000,53.359500,10,28000,**279**,**0**

  This measurement is not of interest to us: the bus is close to bus stop 279, but not stopped at it.

- 2013-01-02 08:10:00,**40**,015B1001,0,-6.258200,53.359300,-20,33489,**279**,**1**

  This measurement is of interest to us. Line 40 is stopped at bus stop 279 (although we already found line 40 before, so it only counts once).

- 2013-01-02 08:15:00,**72**,040B1001,1,-6.257000,53.360000,50,33145,**279**,**1**

  This measurement is of interest to us. Line 72 is stopped at bus stop 279.

- 2013-01-02 08:20:00,50,027B1002,0,-6.259500,53.359600,15,28001,**450**,1

  This measurement is not of interest to us: wrong bus stop.

- 2013-01-02 08:30:00,40,015B1001,0,-6.258300,53.359350,-10,33490,**279**,**0**

  This measurement is not of interest to us: the bus is close to bus stop 279, but not stopped at it.

- 2013-01-03 09:00:00,**15**,,0,-6.261000,53.362000,0,33600,**279**,**1**

  This measurement is of interest to us. Line 15 is stopped at bus stop 279 (its empty Bus_Line_Pattern is irrelevant for this exercise).

The dataset A01_micro_dataset_1 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-02 08:00:00,40,015B1001,0,-6.258078,53.359279,30,33488,279,1
- 2013-01-02 08:05:00,50,027B1002,0,-6.259000,53.359500,10,28000,279,0
- 2013-01-02 08:10:00,40,015B1001,0,-6.258200,53.359300,-20,33489,279,1
- 2013-01-02 08:15:00,72,040B1001,1,-6.257000,53.360000,50,33145,279,1
- 2013-01-02 08:20:00,50,027B1002,0,-6.259500,53.359600,15,28001,450,1
- 2013-01-02 08:25:00,90,090B1001,0,-6.260000,53.361000,5,33500,600,1
- 2013-01-02 08:30:00,40,015B1001,0,-6.258300,53.359350,-10,33490,279,0
- 2013-01-02 08:35:00,72,040B1002,1,-6.257100,53.360100,60,33146,279,0
- 2013-01-03 09:00:00,15,,0,-6.261000,53.362000,0,33600,279,1
- 2013-01-03 09:05:00,15,,0,-6.261100,53.362100,5,33600,900,1

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_1

Given a program passing by parameters:

- The bus stop of interest "stop_id" = 279

--- SPARK CORE ---

- solutionRDD:

< 15, 40, 72 >

- solutionRDD printed by the screen:

  15

  40

  72

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=15), Row(busLineID=40), Row(busLineID=72) >

- solutionDF printed by the screen:

  Row(busLineID=15)

  Row(busLineID=40)

  Row(busLineID=72)

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- The bus stop of interest "stop_id" = 279

--- SPARK CORE ---

- solutionRDD:

  < 7, 40 >

- solutionRDD printed by the screen:

  7

  40

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=7), Row(busLineID=40) >

- solutionDF printed by the screen:

  Row(busLineID=7)

  Row(busLineID=40)

EXERCISE 2.

We use to think that each bus line always follows the very same sequence of bus stops. However, this does not need to be the case: a same bus line can be served by several route-pattern variants (Bus_Line_Pattern) throughout the day (e.g., different branches or directions). The operations team wants to check, for a concrete bus line, which pattern variants actually show up in the recorded data, so they can be cross-checked against the official pattern catalogue. The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- The bus line of interest "bus_line_id" (e.g., 40)

Your task is to:

- Compute the distinct, non-empty route-pattern variants (Bus_Line_Pattern) recorded for bus_line_id. Measurements of bus_line_id with an empty pattern ("") must be discarded.

The format of the solution computed must be:

- < P1, P2, ..., Pn >

where:

- Pi is a String representing a route-pattern variant of bus_line_id.
- < P1, P2, ..., Pn > are sorted by increasing (alphabetical) order of Pi.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_2

Let's assume we are interested in "bus_line_id" = 40.

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-02 08:00:00,**40**,**015B1002**,0,-6.258078,53.359279,30,33488,279,1

  This measurement is of interest to us. Line 40 follows the pattern 015B1002.

- 2013-01-02 08:05:00,**40**,,0,-6.259000,53.359500,10,28000,279,0

  This measurement is not of interest to us: empty pattern.

- 2013-01-02 08:10:00,**40**,**015B1001**,0,-6.258200,53.359300,-20,33489,279,1

  This measurement is of interest to us. Line 40 follows the pattern 015B1001.

- 2013-01-02 08:15:00,**72**,040B1001,1,-6.257000,53.360000,50,33145,279,1

This measurement is not of interest to us: wrong bus line.

- 2013-01-02 08:20:00,**40**,**015B1002**,0,-6.259500,53.359600,15,28001,450,1

  This measurement is not of interest to us: we already noticed that line 40 follows the pattern 015B1002.

The dataset A01_micro_dataset_2 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-02 08:00:00,40,015B1002,0,-6.258078,53.359279,30,33488,279,1
- 2013-01-02 08:05:00,40,,0,-6.259000,53.359500,10,28000,279,0
- 2013-01-02 08:10:00,40,015B1001,0,-6.258200,53.359300,-20,33489,279,1
- 2013-01-02 08:15:00,72,040B1001,1,-6.257000,53.360000,50,33145,279,1
- 2013-01-02 08:20:00,40,015B1002,0,-6.259500,53.359600,15,28001,450,1
- 2013-01-02 08:25:00,40,020B1001,0,-6.260000,53.361000,5,33500,600,1
- 2013-01-03 09:00:00,15,,0,-6.261000,53.362000,0,33600,279,1
- 2013-01-03 09:05:00,40,,0,-6.261100,53.362100,5,33600,900,1

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_2

Given a program passing by parameters:

- The bus line of interest "bus_line_id" = 40

--- SPARK CORE ---

- solutionRDD:

  < 015B1001, 015B1002, 020B1001 >

- solutionRDD printed by the screen:

  015B1001

  015B1002

  020B1001

--- SPARK DATAFRAMES ---

- solutionDF:

< Row(busLinePatternID='015B1001'), Row(busLinePatternID='015B1002'), Row(busLinePatternID='020B1001') >

- solutionDF printed by the screen:

  Row(busLinePatternID='015B1001')

  Row(busLinePatternID='015B1002')

  Row(busLinePatternID='020B1001')

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- The bus line of interest "bus_line_id" = 40

--- SPARK CORE ---

- solutionRDD:

  < 00400001, 00400002, 00400003, 00400004, 00401001, 00401002, 00401003, 040B0001, 040B0002, 040B1001, 040D0001, 040D0002, 040D1001, 040D1002 >

- solutionRDD printed by the screen:

  00400001

  00400002

  00400003

  00400004

  00401001

  00401002

  00401003

  040B0001

  040B0002

  040B1001

  040D0001

  040D0002

  040D1001

  040D1002

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLinePatternID='00400001'), Row(busLinePatternID='00400002'), Row(busLinePatternID='00400003'), Row(busLinePatternID='00400004'), Row(busLinePatternID='00401001'), Row(busLinePatternID='00401002'), Row(busLinePatternID='00401003'), Row(busLinePatternID='040B0001'), Row(busLinePatternID='040B0002'), Row(busLinePatternID='040B1001'), Row(busLinePatternID='040D0001'), Row(busLinePatternID='040D0002'), Row(busLinePatternID='040D1001'), Row(busLinePatternID='040D1002') >

- solutionDF printed by the screen:

  Row(busLinePatternID='00400001')

  Row(busLinePatternID='00400002')

  Row(busLinePatternID='00400003')

  Row(busLinePatternID='00400004')

  Row(busLinePatternID='00401001')

  Row(busLinePatternID='00401002')

  Row(busLinePatternID='00401003')

  Row(busLinePatternID='040B0001')

  Row(busLinePatternID='040B0002')

  Row(busLinePatternID='040B1001')

  Row(busLinePatternID='040D0001')

  Row(busLinePatternID='040D0002')

  Row(busLinePatternID='040D1001')

  Row(busLinePatternID='040D1002')

EXERCISE 3.

Bus vehicles are not always on time: sometimes they run ahead of schedule, sometimes behind it. Fleet managers want to know the single most extreme "bus behind schedule" and "bus ahead of schedule" events ever recorded across the entire bus fleet, to investigate possible sensor faults or genuinely exceptional circumstances (e.g., an accident causing a huge delay). The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- No parameters

Your task is to:

- Compute the measurement with maximum delay (i.e., the bus most behind schedule) and the measurement with minimum delay (i.e., the bus most ahead of schedule, with the most negative delay). For each of them, report its bus line, vehicle and delay.

The format of the solution computed must be:

- < (Lmax, Vmax, Dmax), (Lmin, Vmin, Dmin) >

where:

- Lmax is an Integer representing the bus line of the measurement with maximum delay.
- Vmax is an Integer representing the bus vehicle of the measurement with maximum delay.
- Dmax is an Integer representing the maximum delay (in seconds).
- Lmin, Vmin and Dmin are the analogous for the measurement with minimum delay.
- The maximum is always printed first, followed by the minimum. In case of a tie, just report any of the measurements.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_3

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-02 08:00:00,**40**,015B1002,0,-6.258078,53.359279,**300**,**33488**,279,0

  At this point the only delay we have found is 300, so it represents both our maximum and our minimum.

- 2013-01-02 08:05:00,**50**,027B1002,0,-6.259000,53.359500,**-450**,**28000**,279,0

We adjust the minimum to -450 (line 50, vehicle 28000).

- 2013-01-02 08:10:00,**72**,040B1001,1,-6.257000,53.360000,**900**,**33145**,279,1

  We adjust the maximum to 900 (line 72, vehicle 33145).

- 2013-01-02 08:20:00,90,090B1001,0,-6.260000,53.361000,**150**,33500,600,1

  This measurement alters neither the maximum nor the minimum.

The dataset A01_micro_dataset_3 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-02 08:00:00,40,015B1002,0,-6.258078,53.359279,300,33488,279,0
- 2013-01-02 08:05:00,50,027B1002,0,-6.259000,53.359500,-450,28000,279,0
- 2013-01-02 08:10:00,72,040B1001,1,-6.257000,53.360000,900,33145,279,1
- 2013-01-02 08:15:00,40,015B1002,0,-6.258200,53.359300,-700,33488,279,0
- 2013-01-02 08:20:00,90,090B1001,0,-6.260000,53.361000,150,33500,600,1
- 2013-01-02 08:25:00,15,,0,-6.261000,53.362000,-20,33600,900,1

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_3

Given a program passing by parameters:

- No parameters

--- SPARK CORE ---

- solutionRDD:

  < (72, 33145, 900), (40, 33488, -700) >

- solutionRDD printed by the screen:

  (72, 33145, 900)

  (40, 33488, -700)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=72, vehicleID=33145, delay=900), Row(busLineID=40, vehicleID=33488, delay=-700) >

- solutionDF printed by the screen:

Row(busLineID=72, vehicleID=33145, delay=900)

Row(busLineID=40, vehicleID=33488, delay=-700)

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- No parameters

--- SPARK CORE ---

- solutionRDD:

  < (63, 33197, 116122), (66, 33440, -15045) >

- solutionRDD printed by the screen:

  (63, 33197, 116122)

  (66, 33440, -15045)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=63, vehicleID=33197, delay=116122), Row(busLineID=66, vehicleID=33440, delay=-15045) >

- solutionDF printed by the screen:

  Row(busLineID=63, vehicleID=33197, delay=116122)

  Row(busLineID=66, vehicleID=33440, delay=-15045)

EXERCISE 4.

It is the weekend, and you are planning a leisure trip around Dublin. There is a shortlist of bus lines that can take you to your destination, and you are flexible with the hour you travel at. The startup OptimiseYourJourney wants to recommend you the most reliable option, rather than just the fastest one.

For a bus measurement, a fixed value or delay limit can be used to determine whether the bus is on schedule: if the delay value of the measurement is within the range (-1.0) * delay_limit <= delay <= delay_limit, we say the bus is *on schedule*.

You want to compute, for each bus line of the shortlist, the percentage of its weekend measurements (within the hours you are willing to travel at) that are on schedule. The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- A list of bus lines "line_list" (e.g., [40, 72])

- A list of hours "hours_list" (e.g., ["08", "09"])

- The delay threshold or "delay_limit" (e.g., 60)

Your task is to:

- Compute, per bus line, the percentage of measurements reporting the bus to be on schedule. Consider only the measurements that (a) take place on a weekend day (Saturday or Sunday), (b) belong to a bus line in line_list, and (c) take place in an hour interval of hours_list. Any measurement not satisfying (a), (b) and (c) must be discarded.

The format of the solution computed must be:

- < (L1, P1, T1, O1), (L2, P2, T2, O2), ..., (Ln, Pn, Tn, On) >

where:

- Li is an Integer representing the bus line (e.g., 40).
- Pi is a Float representing the percentage of on schedule measurements, with 2 decimal float point format (e.g., 37.81).
- Ti is an Integer representing the total number of measurements of the bus line (e.g., 34814).
- Oi is an Integer representing the total number of on schedule measurements of the bus line (e.g., 13162).
- Bus lines of line_list with no measurement of interest do not appear in the solution.
- < (L1, P1, T1, O1), ..., (Ln, Pn, Tn, On) > are sorted by decreasing order of Pi. Any tie in percentage is broken by sorting the entries by increasing Li order.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_4

Let's assume we are interested in "line_list" = [40, 72], "hours_list" = ["08", "09"] and "delay_limit" = 60 (60 seconds). Please note 2013-01-05 is a Saturday and 2013-01-07 is a Monday.

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- **2013-01-05 08:10:00**,**40**,015B1001,0,-6.258000,53.359000,**30**,33488,279,0

  This measurement is of interest to us. Line 40 on Saturday at 08:10. It is 30 seconds late, so it is on schedule.

- **2013-01-05 08:40:00**,**40**,015B1001,0,-6.258000,53.359000,**90**,33489,279,0

  This measurement is of interest to us. Line 40 on Saturday at 08:40. It is 90 seconds late, so it is not on schedule.

- **2013-01-07** 08:10:00,40,015B1001,0,-6.258000,53.359000,10,33488,279,0

  This measurement is not of interest to us: weekday (Monday).

- 2013-01-05 **10:10:00**,40,015B1001,0,-6.258000,53.359000,10,33488,279,0

  This measurement is not of interest to us: hour not in hours_list.

- 2013-01-05 08:20:00,**90**,090B1001,0,-6.260000,53.361000,10,33500,600,0

  This measurement is not of interest to us: bus line not in line_list.

The dataset A01_micro_dataset_4 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-05 08:10:00,40,015B1001,0,-6.258000,53.359000,30,33488,279,0
- 2013-01-05 08:40:00,40,015B1001,0,-6.258000,53.359000,90,33489,279,0
- 2013-01-05 09:05:00,72,040B1001,1,-6.257000,53.360000,-20,33145,450,0
- 2013-01-05 09:15:00,72,040B1001,1,-6.257000,53.360000,10,33146,450,0
- 2013-01-05 09:45:00,72,040B1001,1,-6.257000,53.360000,200,33147,450,0
- 2013-01-07 08:10:00,40,015B1001,0,-6.258000,53.359000,10,33488,279,0
- 2013-01-05 10:10:00,40,015B1001,0,-6.258000,53.359000,10,33488,279,0
- 2013-01-05 08:20:00,90,090B1001,0,-6.260000,53.361000,10,33500,600,0

In total, line 40 has 2 measurements of interest, 1 of them on schedule (50.0%), and line 72 has 3 measurements of interest, 2 of them on schedule (66.67%).

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_4

Given a program passing by parameters:

- A list of bus lines "line_list" = [40, 72]

- A list of hours "hours_list" = ["08", "09"]

- The delay threshold or "delay_limit" = 60

--- SPARK CORE ---

- solutionRDD:

  < (72, 66.67, 3, 2), (40, 50.0, 2, 1) >

- solutionRDD printed by the screen:

  (72, 66.67, 3, 2)

  (40, 50.0, 2, 1)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=72, percentage=66.67, total=3, onTime=2), Row(busLineID=40, percentage=50.0, total=2, onTime=1) >

- solutionDF printed by the screen:

  Row(busLineID=72, percentage=66.67, total=3, onTime=2)

  Row(busLineID=40, percentage=50.0, total=2, onTime=1)

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- A list of bus lines "line_list" = [40, 72]

- A list of hours "hours_list" = ["08", "09"]

- The delay threshold or "delay_limit" = 60

--- SPARK CORE ---

- solutionRDD:

< (40, 37.81, 34814, 13162) >

- solutionRDD printed by the screen:

  (40, 37.81, 34814, 13162)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=40, percentage=37.81, total=34814, onTime=13162) >

- solutionDF printed by the screen:

  Row(busLineID=40, percentage=37.81, total=34814, onTime=13162)

Please note line 72 does not appear in the solution, as it has no measurement of interest in the entire dataset.

EXERCISE 5.

Each bus vehicle reports its position periodically through a GPS feed. However, such feed can drop out for a while: the bus vehicle keeps running its route, but no measurements are reported for several minutes. We call this a *ghost bus*, as its live map would show a constant position of the bus during that time.

For each bus vehicle, its measurements in chronological order can be used to detect such gaps: if the elapsed time between two consecutive measurements of the same bus vehicle is greater than a gap limit, we say there is a *ghost gap* between them.

You want to compute the ghost gaps of the bus vehicles serving a concrete bus line in a concrete day, so that the support staff can investigate its reliability. The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- The bus line "bus_line_id" (e.g., 40)

- The concrete day being selected "day_picked" (e.g., "2013-01-07")

- The gap threshold or "gap_limit_seconds" (e.g., 600)

Your task is to:

- Compute the ghost gaps of the bus vehicles serving bus_line_id at day_picked. For each bus vehicle, consider its measurements in chronological order: whenever the elapsed time between two consecutive measurements is strictly greater than gap_limit_seconds, report a ghost gap for them. A bus vehicle with no ghost gaps does not appear in the solution.

The format of the solution computed must be:

- < (V1, S1, E1, G1), (V2, S2, E2, G2), ..., (Vn, Sn, En, Gn) >

where:

- Vi is an Integer representing the bus vehicle (e.g., 33488).
- Si is a String representing the date of the measurement starting the gap (e.g., "2013-01-07 08:05:00").
- Ei is a String representing the date of the measurement ending the gap (e.g., "2013-01-07 08:20:00").
- Gi is an Integer representing the length of the gap, measured in seconds (e.g., 900).
- < (V1, S1, E1, G1), ..., (Vn, Sn, En, Gn) > are sorted by increasing order of Si.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_5

Let's assume we are interested in "bus_line_id" = 40, "day_picked" = "2013-01-07" and "gap_limit_seconds" = 600 (ten minutes, or 600 seconds).

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-07 **08:00:00**,40,015B1001,0,-6.258000,53.359000,10,**33488**,279,0

  This measurement is of interest to us. It is the first measurement of vehicle 33488 on 2013-01-07.

- 2013-01-07 **08:05:00**,40,015B1001,0,-6.258100,53.359100,10,**33488**,280,0

  This measurement is of interest to us. Vehicle 33488 reports 300 seconds after its previous measurement: no ghost gap.

- 2013-01-07 **08:20:00**,40,015B1001,0,-6.258200,53.359200,10,**33488**,281,0

  This measurement is of interest to us. Vehicle 33488 reports 900 seconds after its previous measurement (08:05:00), which is greater than 600: ghost gap.

- 2013-01-07 **08:25:00**,40,015B1001,0,-6.258300,53.359300,10,**33488**,282,0

  This measurement is of interest to us. Vehicle 33488 reports 300 seconds after its previous measurement: no ghost gap.

- 2013-01-07 **09:00:00**,40,015B1002,0,-6.259000,53.360000,0,**33489**,300,0

  This measurement is of interest to us. It is the first measurement of vehicle 33489 on 2013-01-07.

- 2013-01-07 **09:02:00**,40,015B1002,0,-6.259100,53.360100,0,**33489**,301,0

  This measurement is of interest to us. Vehicle 33489 reports 120 seconds after its previous measurement: no ghost gap.

- 2013-01-07 **07:00:00**,40,015B1003,0,-6.256000,53.358000,0,**33491**,200,0

  This measurement is of interest to us. It is the first measurement of vehicle 33491 on 2013-01-07.

- 2013-01-07 **07:20:00**,40,015B1003,0,-6.256100,53.358100,0,**33491**,201,0

  This measurement is of interest to us. Vehicle 33491 reports 1200 seconds after its previous measurement (07:00:00), which is greater than 600: ghost gap.

- 2013-01-07 **07:25:00**,40,015B1003,0,-6.256200,53.358200,0,**33491**,202,0

  This measurement is of interest to us. Vehicle 33491 reports 300 seconds after its previous measurement: no ghost gap.

- 2013-01-07 08:10:00,**72**,040B1001,0,-6.257000,53.360000,0,33145,279,0

This measurement is not of interest to us: wrong bus line.

- **2013-01-08** 08:00:00,40,015B1001,0,-6.258000,53.359000,0,33490,279,0

  This measurement is not of interest to us: wrong day.

The dataset A01_micro_dataset_5 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-07 08:00:00,40,015B1001,0,-6.258000,53.359000,10,33488,279,0
- 2013-01-07 08:05:00,40,015B1001,0,-6.258100,53.359100,10,33488,280,0
- 2013-01-07 08:20:00,40,015B1001,0,-6.258200,53.359200,10,33488,281,0
- 2013-01-07 08:25:00,40,015B1001,0,-6.258300,53.359300,10,33488,282,0
- 2013-01-07 09:00:00,40,015B1002,0,-6.259000,53.360000,0,33489,300,0
- 2013-01-07 09:02:00,40,015B1002,0,-6.259100,53.360100,0,33489,301,0
- 2013-01-07 07:00:00,40,015B1003,0,-6.256000,53.358000,0,33491,200,0
- 2013-01-07 07:20:00,40,015B1003,0,-6.256100,53.358100,0,33491,201,0
- 2013-01-07 07:25:00,40,015B1003,0,-6.256200,53.358200,0,33491,202,0
- 2013-01-07 08:10:00,72,040B1001,0,-6.257000,53.360000,0,33145,279,0
- 2013-01-08 08:00:00,40,015B1001,0,-6.258000,53.359000,0,33490,279,0
- 2013-01-08 08:20:00,40,015B1001,0,-6.258000,53.359000,0,33490,280,0

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_5

Given a program passing by parameters:

- The bus line "bus_line_id" = 40

- The concrete day being selected "day_picked" = "2013-01-07"

- The gap threshold or "gap_limit_seconds" = 600

--- SPARK CORE ---

- solutionRDD:

  < (33491, '2013-01-07 07:00:00', '2013-01-07 07:20:00', 1200), (33488, '2013-01-07 08:05:00', '2013-01-07 08:20:00', 900) >

- solutionRDD printed by the screen:

(33491, '2013-01-07 07:00:00', '2013-01-07 07:20:00', 1200)

(33488, '2013-01-07 08:05:00', '2013-01-07 08:20:00', 900)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(vehicleID=33491, gapStart='2013-01-07 07:00:00', gapEnd='2013-01-07 07:20:00', gapSeconds=1200), Row(vehicleID=33488, gapStart='2013-01-07 08:05:00', gapEnd='2013-01-07 08:20:00', gapSeconds=900) >

- solutionDF printed by the screen:

  Row(vehicleID=33491, gapStart='2013-01-07 07:00:00', gapEnd='2013-01-07 07:20:00', gapSeconds=1200)

  Row(vehicleID=33488, gapStart='2013-01-07 08:05:00', gapEnd='2013-01-07 08:20:00', gapSeconds=900)

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- The bus line "bus_line_id" = 40

- The concrete day being selected "day_picked" = "2013-01-07"

- The gap threshold or "gap_limit_seconds" = 600

--- SPARK CORE ---

- solutionRDD:

  < (33154, '2013-01-07 00:01:04', '2013-01-07 06:00:06', 21542), (33161, '2013-01-07 00:24:06', '2013-01-07 06:15:52', 21106), …, (33157, '2013-01-07 22:12:16', '2013-01-07 22:58:02', 2746) >

- solutionRDD printed by the screen:

  (33154, '2013-01-07 00:01:04', '2013-01-07 06:00:06', 21542)

  (33161, '2013-01-07 00:24:06', '2013-01-07 06:15:52', 21106)

  …

  (33157, '2013-01-07 22:12:16', '2013-01-07 22:58:02', 2746)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(vehicleID=33154, gapStart='2013-01-07 00:01:04', gapEnd='2013-01-07 06:00:06', gapSeconds=21542), Row(vehicleID=33161, gapStart='2013-01-07

00:24:06', gapEnd='2013-01-07 06:15:52', gapSeconds=21106), …, Row(vehicleID=33157, gapStart='2013-01-07 22:12:16', gapEnd='2013-01-07 22:58:02', gapSeconds=2746) >

- solutionDF printed by the screen:

  Row(vehicleID=33154, gapStart='2013-01-07 00:01:04', gapEnd='2013-01-07 06:00:06', gapSeconds=21542)

  Row(vehicleID=33161, gapStart='2013-01-07 00:24:06', gapEnd='2013-01-07 06:15:52', gapSeconds=21106)

  …

  Row(vehicleID=33157, gapStart='2013-01-07 22:12:16', gapEnd='2013-01-07 22:58:02', gapSeconds=2746)

EXERCISE 6.

"*Stopping advertising to save money is like stopping your watch to save time.*" – Henry Ford. The marketing department of OptimiseYourJourney wants to send a promotional notification to the users travelling on a concrete bus line. They want to choose the bus line(s) stopping at the highest amount of different bus stops.

A *static approach* could be to consider the entire dataset. Instead, they prefer a *dynamic approach*, where the analysis focuses on a small time window, based on a given time and a seconds horizon after it.

You want to compute the bus line(s) reaching the highest amount of different bus stops for a time window. The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- The start time for the time window "current_time" (e.g., "2013-01-09 06:00:00")

- The length of the time window "seconds_horizon" (e.g., 1800)

Your task is to:

- Compute the bus line(s) stopping at the highest amount of different bus stops during the time window [current_time, current_time + seconds_horizon). Consider only measurements where the bus is stopped at the bus stop (At_Stop == 1). For a given bus line, consider only different bus stops (if a bus line stops twice or more at the same bus stop during the time window, you only count it once).

The format of the solution computed must be:

- < (L1, S1), (L2, S2), ..., (Ln, Sn) >

where:

- Li is an Integer representing the bus line (e.g., 72).
- Si is an Integer list representing the bus stops the bus line stops at during the time window, sorted in increasing order (e.g., [279, 900, 1000]).
- In case multiple bus lines have the very same amount of max bus stops, sort them in increasing bus line order.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_6

Let's assume we are interested in "current_time" = "2013-01-09 06:00:00" and "seconds_horizon" = 1800 (30 minutes), i.e., in the time window [06:00:00, 06:30:00).

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-09 **06:05:00**,**72**,040B1001,0,-6.257000,53.360000,0,33145,**279**,**1**

  This measurement is of interest to us. Line 72 stops at bus stop 279.

- 2013-01-09 **06:10:00**,**72**,040B1001,0,-6.257000,53.360000,0,33146,**900**,**1**

  This measurement is of interest to us. Line 72 stops at bus stop 900.

- 2013-01-09 **06:15:00**,**72**,040B1001,0,-6.257000,53.360000,0,33147,**1000**,**1**

  This measurement is of interest to us. Line 72 stops at bus stop 1000.

- 2013-01-09 **06:20:00**,**72**,040B1001,0,-6.257000,53.360000,0,33148,**279**,**1**

  This measurement is of interest to us, but line 72 already stopped at bus stop 279 before, so it only counts once.

- 2013-01-09 **06:05:00**,**40**,015B1001,0,-6.258000,53.359000,0,33488,**279**,**1**

  This measurement is of interest to us. Line 40 stops at bus stop 279.

- 2013-01-09 **06:10:00**,**40**,015B1001,0,-6.258000,53.359000,0,33489,**450**,**1**

  This measurement is of interest to us. Line 40 stops at bus stop 450.

- 2013-01-09 **06:12:00**,**50**,027B1001,0,-6.259000,53.359500,0,28000,**279**,**1**

  This measurement is of interest to us. Line 50 stops at bus stop 279.

- 2013-01-09 06:07:00,72,040B1001,0,-6.257000,53.360000,0,33149,600,**0**

  This measurement is not of interest to us: the bus is not stopped at the bus stop.

- 2013-01-09 **06:45:00**,72,040B1001,0,-6.257000,53.360000,0,33150,700,1

  This measurement is not of interest to us: after the time window.

- 2013-01-09 **05:50:00**,72,040B1001,0,-6.257000,53.360000,0,33151,800,1

  This measurement is not of interest to us: before the time window.

The dataset A01_micro_dataset_6 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-09 06:05:00,72,040B1001,0,-6.257000,53.360000,0,33145,279,1
- 2013-01-09 06:10:00,72,040B1001,0,-6.257000,53.360000,0,33146,900,1
- 2013-01-09 06:15:00,72,040B1001,0,-6.257000,53.360000,0,33147,1000,1
- 2013-01-09 06:20:00,72,040B1001,0,-6.257000,53.360000,0,33148,279,1

- 2013-01-09 06:05:00,40,015B1001,0,-6.258000,53.359000,0,33488,279,1
- 2013-01-09 06:10:00,40,015B1001,0,-6.258000,53.359000,0,33489,450,1
- 2013-01-09 06:12:00,50,027B1001,0,-6.259000,53.359500,0,28000,279,1
- 2013-01-09 06:07:00,72,040B1001,0,-6.257000,53.360000,0,33149,600,0
- 2013-01-09 06:45:00,72,040B1001,0,-6.257000,53.360000,0,33150,700,1
- 2013-01-09 05:50:00,72,040B1001,0,-6.257000,53.360000,0,33151,800,1

In total, line 72 stops at 3 different bus stops < 279, 900, 1000 >, line 40 at 2 < 279, 450 > and line 50 at 1 < 279 >. Thus, line 72 is the one stopping at the highest amount of different bus stops.

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_6

Given a program passing by parameters:

- The start time for the time window "current_time" = "2013-01-09 06:00:00"

- The length of the time window "seconds_horizon" = 1800

--- SPARK CORE ---

- solutionRDD:

  < (72, [279, 900, 1000]) >

- solutionRDD printed by the screen:

  (72, [279, 900, 1000])

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=72, stops=[279, 900, 1000]) >

- solutionDF printed by the screen:

  Row(busLineID=72, stops=[279, 900, 1000])

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- The start time for the time window "current_time" = "2013-01-09 06:00:00"

- The length of the time window "seconds_horizon" = 1800

--- SPARK CORE ---

- solutionRDD:

  < (27, [298, 395, 496, …, 6130, 7550]), (40, [7, 15, 49, …, 7125, 7344]) >

- solutionRDD printed by the screen:

  (27, [298, 395, 496, …, 6130, 7550])

  (40, [7, 15, 49, …, 7125, 7344])

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(busLineID=27, stops=[298, 395, 496, …, 6130, 7550]), Row(busLineID=40, stops=[7, 15, 49, …, 7125, 7344]) >

- solutionDF printed by the screen:

  Row(busLineID=27, stops=[298, 395, 496, …, 6130, 7550])

  Row(busLineID=40, stops=[7, 15, 49, …, 7125, 7344])

Please note that, in the entire dataset, lines 27 and 40 tie, each of them stopping at 64 different bus stops. The complete lists of bus stops are available in the files of the folder my_results.

EXERCISE 7.

The operations team wants a fleet-wide profile of the *typical driving speed* of the bus vehicles in a concrete day, so as to flag unusually slow bus vehicles (possible mechanical issues).

For each bus vehicle, its measurements in chronological order can be used to compute its speed: for each pair of consecutive measurements, the distance between their positions divided by the elapsed time between them gives the speed of the bus between the two measurements. The distance between two positions (longitude, latitude) is computed with the *Haversine formula*, using a mean Earth radius of 6371.0 km.

However, GPS measurements are not always accurate. Sometimes two consecutive measurements imply a physically implausible speed (e.g., hundreds of kilometres covered in a few seconds). Any pair of consecutive measurements implying a speed greater than a maximum speed accepted is considered GPS noise and must be discarded.

You want to compute a histogram with the average speed of the bus vehicles in a concrete day. The dataset you got in your hands certainly allows you to infer this information; so let's analyse it to find out.

EXERCISE: FORMAL DEFINITION

Given a program passing by parameters:

- The bucket or speed range interval size (measured in km/h), or "bucket_size_kmh" (e.g., 5.0)

- The maximum speed accepted (measured in metres per second), or "max_speed_accepted" (e.g., 28.0)

- The day selected, or "day_picked" (e.g., "2013-01-07")

Your task is to:

- Compute the histogram of the average speed of the bus vehicles at day_picked. For each bus vehicle, consider its measurements in chronological order, and compute the speed for each pair of consecutive measurements. Discard any pair with a speed greater than max_speed_accepted (in m/s). For the remaining (valid) pairs, convert the speed to km/h. The average speed of the bus vehicle is the mean of the speeds of its valid pairs. Bus vehicles with no valid pairs (e.g., with a single measurement in the day) are discarded. Finally, assign each bus vehicle to the bucket i covering its average speed, i.e., [i * bucket_size_kmh, (i+1) * bucket_size_kmh), and count the amount of bus vehicles per bucket.

The format of the solution computed must be:

- < (I1, R1, N1), (I2, R2, N2), ..., (In, Rn, Nn) >

where:

- Ii is an Integer representing the index or bucket ID (e.g., 1).

- Ri is a String representing the speed range interval (in km/h) for the bucket, with format "lo_hi" (e.g., "5.0_10.0").
- Ni is an Integer representing the number of bus vehicles with an average speed within the bucket (e.g., 4).
- Buckets with no bus vehicles do not appear in the solution.
- < (I1, R1, N1), ..., (In, Rn, Nn) > are sorted by increasing order of Ii.

EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_7

Let's assume we are interested in "bucket_size_kmh" = 5.0, "max_speed_accepted" = 28.0 and "day_picked" = "2013-01-07".

Let's first consider the following measurements separately to better understand the problem:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-07 **08:00:00**,40,015B1001,0,-6.260000,**53.350000**,10,**33488**,279,0

  This measurement is of interest to us. It is the first measurement of vehicle 33488 on 2013-01-07.

- 2013-01-07 **08:10:00**,40,015B1001,0,-6.260000,**53.360000**,10,**33488**,280,0

  This measurement is of interest to us. Vehicle 33488 travels 0.01 degrees of latitude (~1.1119 km) in 10 minutes: ~1.85 m/s, which is a valid pair, with a speed of ~6.6717 km/h.

- 2013-01-07 **08:22:00**,40,015B1001,0,-6.260000,**53.370000**,10,**33488**,281,0

  This measurement is of interest to us. Vehicle 33488 travels ~1.1119 km in 12 minutes: a valid pair with a speed of ~5.5597 km/h. Let's also assume there are no further measurements for bus vehicle 33488. We can conclude the average speed of vehicle 33488 is ~6.1157 km/h, so it belongs to bucket 1.

- 2013-01-07 **09:00:00**,72,040B1001,0,-6.260000,**53.300000**,10,**28000**,300,0

  This measurement is of interest to us. It is the first measurement of vehicle 28000 on 2013-01-07.

- 2013-01-07 **09:05:00**,72,040B1001,0,-6.260000,**53.310000**,10,**28000**,301,0

  This measurement is of interest to us. Vehicle 28000 travels ~1.1119 km in 5 minutes: a valid pair with a speed of ~13.3434 km/h. Let's also assume there are no further measurements for bus vehicle 33488. We can conclude the average speed of vehicle 28000 is ~13.3434 km/h, so it belongs to bucket 2.

- 2013-01-07 **08:00:00**,50,027B1001,0,-6.260000,**53.300000**,10,**33490**,400,0

  This measurement is of interest to us. It is the first measurement of vehicle 33490 on 2013-01-07.

- 2013-01-07 **08:00:05**,50,027B1001,0,-6.260000,**53.400000**,10,**33490**,401,0

  This measurement is of interest to us. Vehicle 33490 travels 0.1 degrees of latitude (~11.1195 km) in only 5 seconds: ~2223.9 m/s, which is greater than max_speed_accepted. The pair is discarded as GPS noise. Let's also assume there are no further measurements for bus vehicle 33490, so no results are reported for it.

- **2013-01-08** 08:00:00,40,015B1001,0,-6.260000,53.350000,10,33491,279,0

  This measurement is not of interest to us: wrong day.

- **2013-01-08** 08:10:00,40,015B1001,0,-6.260000,53.360000,10,33491,280,0

  This measurement is not of interest to us: wrong day.

The dataset A01_micro_dataset_7 contains just the following measurements:

Date,Bus_Line,Bus_Line_Pattern,Congestion,Longitude,Latitude,Delay,Vehicle,Closer_Stop, At_Stop

- 2013-01-07 08:00:00,40,015B1001,0,-6.260000,53.350000,10,33488,279,0
- 2013-01-07 08:10:00,40,015B1001,0,-6.260000,53.360000,10,33488,280,0
- 2013-01-07 08:22:00,40,015B1001,0,-6.260000,53.370000,10,33488,281,0
- 2013-01-07 09:00:00,72,040B1001,0,-6.260000,53.300000,10,28000,300,0
- 2013-01-07 09:05:00,72,040B1001,0,-6.260000,53.310000,10,28000,301,0
- 2013-01-07 08:00:00,50,027B1001,0,-6.260000,53.300000,10,33490,400,0
- 2013-01-07 08:00:05,50,027B1001,0,-6.260000,53.400000,10,33490,401,0
- 2013-01-08 08:00:00,40,015B1001,0,-6.260000,53.350000,10,33491,279,0
- 2013-01-08 08:10:00,40,015B1001,0,-6.260000,53.360000,10,33491,280,0

SOLUTION EXAMPLE 1 - SMALL DATASET – A01_micro_dataset_7

Given a program passing by parameters:

- The bucket size "bucket_size_kmh" = 5.0

- The maximum speed accepted "max_speed_accepted" = 28.0

- The day selected "day_picked" = "2013-01-07"

--- SPARK CORE ---

- solutionRDD:

  < (1, '5.0_10.0', 1), (2, '10.0_15.0', 1) >

- solutionRDD printed by the screen:

  (1, '5.0_10.0', 1)

  (2, '10.0_15.0', 1)

--- SPARK DATAFRAMES ---

- solutionDF:

  < Row(bucketID=1, bucketRange='5.0_10.0', numVehicles=1), Row(bucketID=2, bucketRange='10.0_15.0', numVehicles=1) >

- solutionDF printed by the screen:

  Row(bucketID=1, bucketRange='5.0_10.0', numVehicles=1)

  Row(bucketID=2, bucketRange='10.0_15.0', numVehicles=1)

SOLUTION EXAMPLE 2 - ENTIRE DATASET – my_dataset_complete

Given a program passing by parameters:

- The bucket size "bucket_size_kmh" = 5.0

- The maximum speed accepted "max_speed_accepted" = 28.0

- The day selected "day_picked" = "2013-01-07"

--- SPARK CORE ---

- solutionRDD:

  < (0, '0.0_5.0', 1), (1, '5.0_10.0', 4), (2, '10.0_15.0', 299), (3, '15.0_20.0', 455), (4, '20.0_25.0', 56), (5, '25.0_30.0', 5), (6, '30.0_35.0', 1) >

- solutionRDD printed by the screen:

  (0, '0.0_5.0', 1)

  (1, '5.0_10.0', 4)

  (2, '10.0_15.0', 299)

  (3, '15.0_20.0', 455)

  (4, '20.0_25.0', 56)

  (5, '25.0_30.0', 5)

  (6, '30.0_35.0', 1)

--- SPARK DATAFRAMES ---

- solutionDF:

< Row(bucketID=0, bucketRange='0.0_5.0', numVehicles=1), Row(bucketID=1, bucketRange='5.0_10.0', numVehicles=4), Row(bucketID=2, bucketRange='10.0_15.0', numVehicles=299), Row(bucketID=3, bucketRange='15.0_20.0', numVehicles=455), Row(bucketID=4, bucketRange='20.0_25.0', numVehicles=56), Row(bucketID=5, bucketRange='25.0_30.0', numVehicles=5), Row(bucketID=6, bucketRange='30.0_35.0', numVehicles=1) >

- solutionDF printed by the screen:

  Row(bucketID=0, bucketRange='0.0_5.0', numVehicles=1)

  Row(bucketID=1, bucketRange='5.0_10.0', numVehicles=4)

  Row(bucketID=2, bucketRange='10.0_15.0', numVehicles=299)

  Row(bucketID=3, bucketRange='15.0_20.0', numVehicles=455)

  Row(bucketID=4, bucketRange='20.0_25.0', numVehicles=56)

  Row(bucketID=5, bucketRange='25.0_30.0', numVehicles=5)

  Row(bucketID=6, bucketRange='30.0_35.0', numVehicles=1)

EXERCISE 8.

There are many other publicly available transportation-related datasets out there (they can be focused on cars, buses, trains, taxis, bikes, scooters, etc).

Survey these alternative datasets, and select a dataset of interest to you:

- A publicly available transportation-related dataset (different from the Dublin Bus dataset).

Compare and contrast the new dataset w.r.t. the Dublin Bus dataset, including:

- A brief description of the dataset.
- Its main characteristics: URL for accessing to it, size, format of the data, etc.

Justify the use of a novel-use case using the aforementioned novel dataset.

- Explain the purpose, design and implementation of such novel use-case.
- For the implementation, use only Spark Core or Spark DataFrames. Justify your choice of technology.